ELK 346 E
Data Acquisition & Embedded Systems

2023 – 2024 Spring Semester
Dr. Aydın Tarık Zengin
(tarik.zengin@itu.edu.tr)

Topics

- Sensors
- Filters, ADCs, DACs
- Sampling
- Fundamental Digital Electronics
- Basics of Microcontrollers – Registers, ALU
- ARM - Introduction
- ARM – Registers
- Embedded Programming

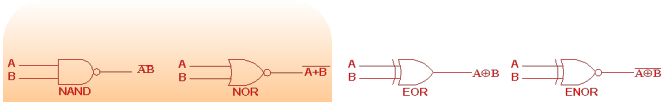
Basics of Digital Design

2 Input AND gate		
A	B	A.B
0	0	0
0	1	0
1	0	0
1	1	1

2 Input OR gate		
A	B	A+B
0	0	0
0	1	1
1	0	1
1	1	1

NOT gate	
A	$\overline{A}$
0	1
1	0

Basics of Digital Design



2 Input NAND gate		
A	B	$\overline{A.B}$
0	0	1
0	1	1
1	0	1
1	1	0

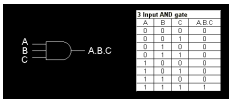
2 Input NOR gate		
A	B	$\overline{A+B}$
0	0	1
0	1	0
1	0	0
1	1	0

2 Input EXOR gate		
A	B	A⊕B
0	0	0
0	1	1
1	0	1
1	1	0

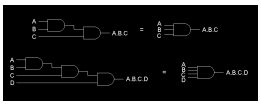
2 Input EXNOR gate		
A	B	$\overline{A⊕B}$
0	0	1
0	1	0
1	0	0
1	1	1

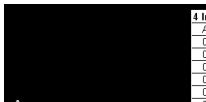
Universal Gates

Multiple Input Gates



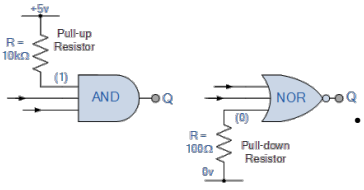
3 Input AND gate			
A	B	C	A.B.C
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1





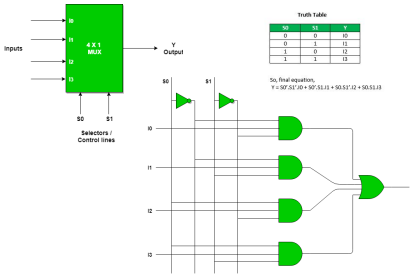
4 Input AND gate				
A	B	C	D	A.B.C.D
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

Pull-up / Pull-down Resistors



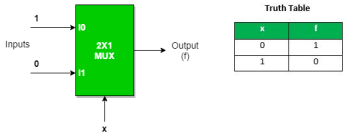
- Unused inputs to the gates must be connected directly to either a logic level “1” or a logic level “0” by means of a suitable “Pull-up” or “Pull-down” resistor
- This will prevent the unused input to the gate from “floating” about and producing false switching of the gate and circuit.

Multiplexers



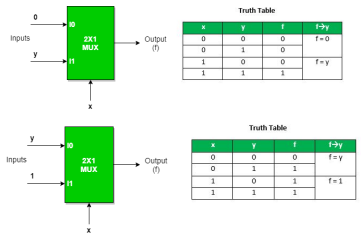
- A.K.A., Data n selector, parallel to serial convertor, many to one circuit, universal logic circuit
- For 2^n input lines, n selection lines are required

Design with Mux



- NOT Gate with Mux
- Y = x'.1 + x.0 = x'

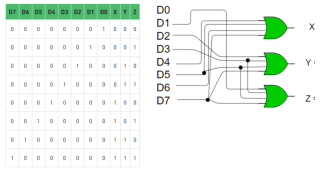
Design with Mux



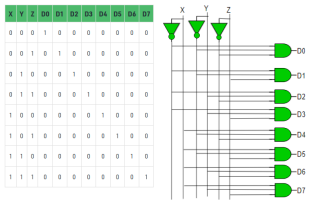
- AND Gate with Mux
- OR Gate with Mux
- All other functions can be implemented by using mux

Encoder / Decoder

- Encoder

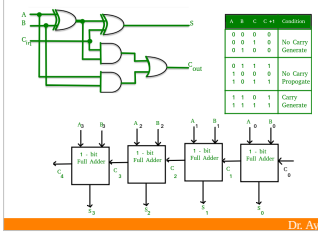


- Decoder

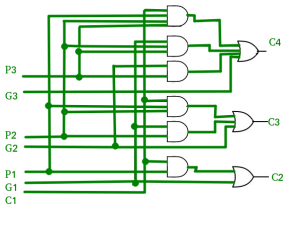


Adders

- Carry generate/propagate



- Carry Look-Ahead Adder



Sequential Circuits

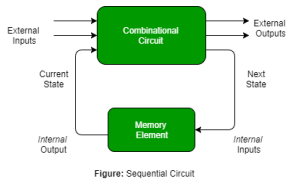


Figure: Sequential Circuit

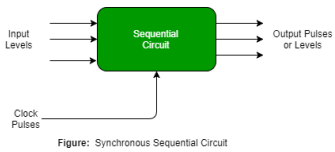
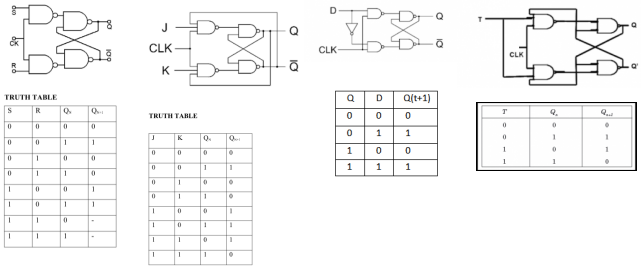


Figure: Synchronous Sequential Circuit

Flip-Flops



TRUTH TABLE

S	R	Q_N	Q_{N+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	-
1	1	1	-

TRUTH TABLE

J	K	Q_N	Q_{N+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Q	D	Q(t+1)
0	0	0
0	1	1
1	0	0
1	1	1

T	Q_N	Q_{N+1}
0	0	0
0	1	1
1	0	1
1	1	0

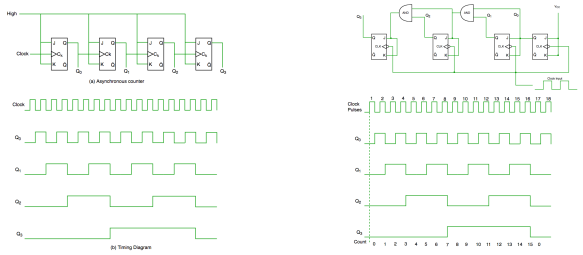
Flip-Flops

- Excitation Table

Q_N	Q_{N+1}	S	R	J	K	D	T
0	0	0	X	0	X	0	0
0	1	1	0	1	X	1	1
1	0	0	1	X	1	0	1
1	1	X	0	X	0	1	0

Counters

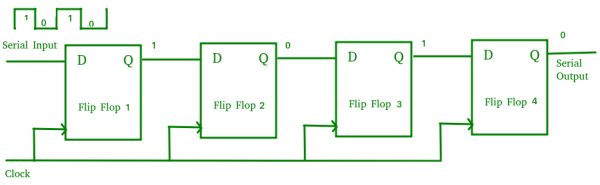
- Asynchronous Counter
- Synchronous Counter



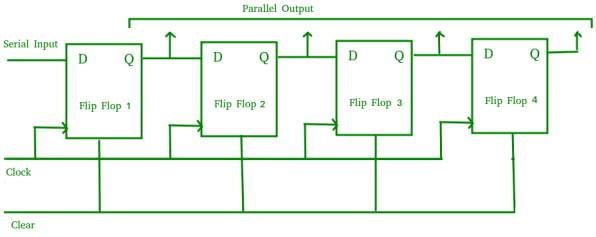
Shift Registers

- Flip-flops store 1 bit data
- To store multiple bits in memory, we need registers with multiple flip-flops
- n flip-flops for n bit data
- Shift left registers / Shift right registers

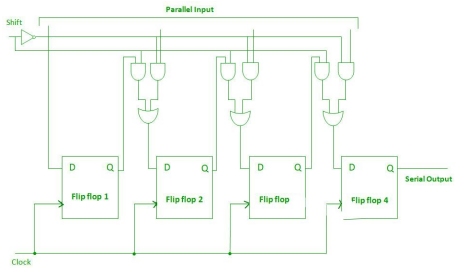
Serial-In Serial-Out Shift Register (SISO)



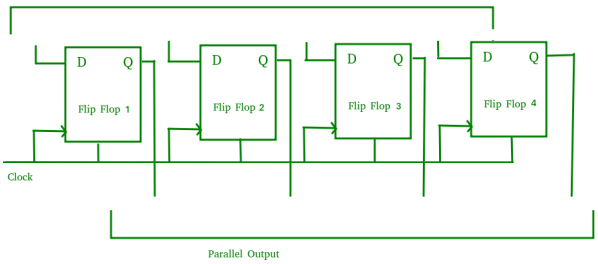
Serial-In Parallel-Out shift Register (SIPO)



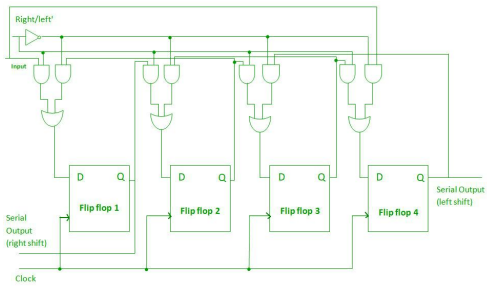
Parallel-In Serial-Out Shift Register (PISO)



Parallel-In Parallel-Out Shift Register (PIPO)



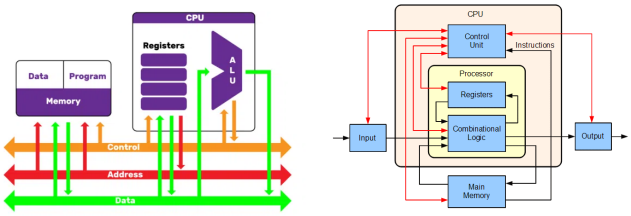
Bidirectional Shift Register



Topics

- Sensors
- Filters, ADCs, DACs
- Sampling
- Fundamental Digital Electronics
- Basics of Microcontrollers – Registers, ALU
- ARM - Introduction
- ARM – Registers
- Embedded Programming

CPU Architecture



Registers

8086

AX (Accumulator) 16 bits divided into two 8-bit registers AH and AL. It is used in arithmetic, logic and data transfer instructions.

BX (Base Register) 16 bits divided into two 8-bit registers BH and BL. BX is an address register. It usually contains a data pointer used for based, based indexed or register indirect addressing.

CX (Count register) 16 bits divided into two 8-bit registers CH and CL. This serves as a loop counter. Program loop constructions are facilitated by it.

DX (Data Register) 16 bits divided into two 8-bit registers DH and DL. It is also used in multiplication and division.

SP (Stack Pointer) This is stack pointer register pointing to program stack. It is used in conjunction with SS for accessing the stack segment. It is of 16 bits. It points to the topmost item of the stack. If the stack is empty the stack pointer will be (FFFE)H.

BP (Base Pointer) Points to data in stack segment. Unlike SP, we can use BP to access data in the other segments. It is of 16 bits. It is primarily used in accessing parameters passed by the stack.

SI (Source Index) Points to memory locations in the data segment addressed by DS. Thus, when we increment the contents of SI, we can easily access consecutive memory locations. It is of 16 bits.

DI (Destination Index) Performs the same function as SI. There is a class of instructions called string operations, that use DI to access the memory locations addressed by ES.

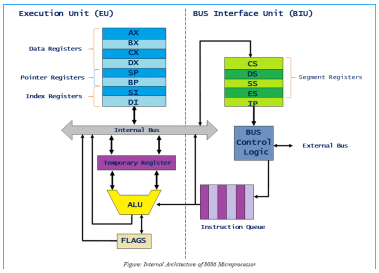


Figure: Internal Architecture of 8086 Microprocessor

Basic Operation – 8-bit ADD

- Let's say we want to add two 8-bit numbers, stored at memory locations 2000H and 2001H in the data segment.
- We can use the MOV instruction to move the values from memory into registers.
- We are using the AL and BL registers to store the input values, but we could use other registers if desired.

```
MOV AL, [DS:2000H]
; Move the value at memory location 2000H into register AL
MOV BL, [DS:2001H]
; Move the value at memory location 2001H into register BL
```


Basic Operation – 8-bit ADD

- We can use the ADD instruction to add the values in the registers together.
- The result of the addition is stored in register AL.
- Then use the MOV instruction to move the result from the register to a memory location.
- The result is stored in memory location 2002H.

```
ADD AL, BL
; Add the value in register BL to the value in register AL
MOV [DS:2002H], AL
; Move the value in register AL to memory location 2002H
```


Basic Operation – 16-bit ADD

- Let's say we want to add two 16-bit numbers, stored at memory locations 2000H and 2002H in the data segment.
- We can use the MOV instruction to move the values from memory into registers.
- We are using the AX and BX registers to store the input values, but we could use other registers if desired.

```
MOV AX, [DS:2000H]
; Move the value at memory location 2000H into register AX
MOV BX, [DS:2002H]
; Move the value at memory location 2002H into register BX
```


Basic Operation – 16-bit ADD

- We can use the ADD instruction to add the values in the registers together.
- The result of the addition is stored in register AX.
- Then use the MOV instruction to move the result from the register to a memory location.
- The result is stored in memory location 2004H.

```
ADD AX, BX
; Add the value in register BL to the value in register AL
MOV [DS:2004H], AX
; Move the value in register AL to memory location 2004H
```


Basic Operation – 16-bit ADD

- Handling the overflow
  - If the result of the addition operation is larger than the maximum value that can be stored in a 16-bit register (FFFFH), an overflow may occur.
  - In this case, the CPU sets the carry flag in the flags register to indicate that an overflow has occurred. The carry flag can be checked later in the program if needed.
  - If we were working with 32-bit numbers, we would need to use two 16-bit registers to store each number and the ADC (add with carry) instruction to handle any carry from the lower word to the higher word during addition.

```
ADD AX, BX
; Add the value in register BL to the value in register AL
MOV [DS:2004H], AX
; Move the value in register AL to memory location 2004H
```


Basic Operation – 16-bit MUL

- Let's say we want to multiply two 16-bit numbers, stored at memory locations 2000H and 2002H in the data segment.
- We can use the MOV instruction to move the values from memory into registers.
- We are using the AX and BX registers to store the input values, but we could use other registers if desired.

```
MOV AX, [DS:2000H]
; Move the value at memory location 2000H into register AX
MOV BX, [DS:2002H]
; Move the value at memory location 2002H into register BX
```

Basic Operation – 16-bit MUL

- MUL instruction to multiply the values in the registers together.
- The result of the multiplication is stored in the DX:AX register pair as a 32-bit value. The higher 16 bits of the result are stored in the DX register and the lower 16 bits are stored in the AX register.
- MOV instruction to move the result from the DX:AX register pair to a memory location.
- The result is stored in memory locations 2004H and 2006H as a 32-bit value.

```
MUL BX
; Multiply the value in register BX by the value in register AX
MOV [DS:2004H], AX
; Move the lower 16 bits of the result from register AX to memory location 2004H
MOV [DS:2006H], DX
; Move the higher 16 bits of the result from register DX to memory location 2006H
```


Basic Operation – Stack

- Take two 8-bit values from memory, store them in the stack, multiply them together, and write the result back to memory
- Load the first value from memory into a register and push it onto the stack
- Load the second value from memory into a register and push it onto the stack

```
MOV AL, [DS:1000H]
; Move the value at memory location 1000H into register AL
PUSH AX
; Push the value in register AX onto the stack
MOV AL, [DS:1001H]
; Move the value at memory location 1001H into register AL
PUSH AX
; Push the value in register AX onto the stack
```


Basic Operation – Stack

- Pop both values from the stack into registers and multiply them together
- We can use the POP instruction to remove the second value from the stack and move it into a register, and then use the POP instruction again to remove the first value from the stack and move it into another register.
- Then we can use the MUL instruction to multiply the values together.
- Store the result in memory

```
POP BX
; Pop the second value from the stack and move it into register BX
POP AX
; Pop the first value from the stack and move it into register AX
MUL BL
; Multiply the values in registers AL and BL and store the result in AX
MOV [DS:1002H], AX
; Move the result from register AX to memory location 1002H
```


Basic Operation - LOOP

- Access the memory locations containing the numbers from 1 to 6, add 2 to each number, and write the updated values back to their original addresses.
- Let's say the array starts at memory location 2000H in the data segment.
- We can use the MOV instruction to load this base address into the Source Index register

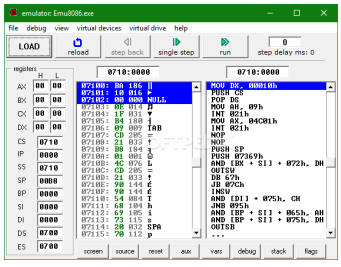
```
MOV CX, 6 ; Set the counter to 6
MOV DI, 2000H ; Set the destination index to the start of the array
MOV AL, 1 ; Start with the value 1
LOOP_START:
  STOSB ; Store the value in AL to memory and increment DI
  INC AL ; Increment the value in AL to get the next number
  LOOP LOOP_START ; Repeat until CX = 0
MOV SI, 2000H ; Load the base address of the array into register SI
```


Basic Operation - LOOP

- Use a loop to iterate through the array
- CX register as a counter to determine how many times to iterate through the array.
- LODSB instruction to load a byte from the memory location pointed to by the index register, and then increment the index register to point to the next byte in the array.
- Add 2 to the value in the AL register (which holds the byte loaded from memory), and write the updated value back to the original memory location using the STOSB instruction.
- Repeat this process until we have processed all 6 numbers in the array.

```
MOV CX, 6 ; Set the counter to 6
LOOP_START:
  LODSB ; Load a byte from memory into AL and increment SI
  ADD AL, 2 ; Add 2 to the value in AL
  STOSB ; Write the updated value back to memory and increment DI
  LOOP LOOP_START ; Repeat until CX = 0
```


8086 Simulation EMU8086



- Try these basic instructions: MOV, XCHG, PUSH, PUSHF, POP, POPF, LAHF, SAHF, ADD, ADC, SUB, SBB, MUL, DIV, INC, DEC, AND, OR, XOR, NOT, SHL, SAL, SHR, SAR, RCL, ROL, RCR, ROR, JMP, LOOP, CMP, JE, JZ, JC, JO, JP, JPE, JNP, JPO, JS

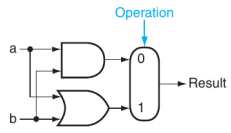
Arithmetic Logic Unit - ALU

- The Arithmetic Logic Unit (ALU) is a component of a CPU (Central Processing Unit) that performs arithmetic and logical operations on data.
- The ALU is responsible for performing operations such as addition, subtraction, multiplication, division, bitwise AND, bitwise OR, and bitwise XOR, etc.
- When a program is executed, the ALU fetches the data from the memory or the registers, performs the required arithmetic or logical operation, and then stores the result back to the memory or the registers.

Arithmetic Logic Unit - ALU

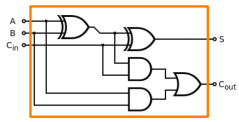
- The basic inputs of an Arithmetic Logic Unit (ALU) include:
  - Data Inputs: These are the values or operands that are input to the ALU for performing arithmetic or logical operations.
  - Control Inputs: These inputs specify the operation to be performed by the ALU. For example, if the control input is set to "add," the ALU will perform an addition operation on the data inputs.
- The basic outputs of an ALU include:
  - Result Output: This is the output of the ALU that represents the result of the arithmetic or logical operation performed on the data inputs.
  - Status Flags: These are output signals that indicate the status of the operation performed by the ALU. For example, the carry flag indicates if there was a carry-out during an addition operation, and the zero flag indicates if the result of an operation is zero.
  - The exact inputs and outputs of an ALU may vary depending on the specific implementation and architecture of the processor. However, the inputs and outputs described above are the most common and fundamental ones found in most ALUs.

1-bit ALU



- Let's design an ALU systematically.
- First step is to design a 1-bit logical unit.
- A multiplexer is needed.
- Operation input decides which gate is going to be used.

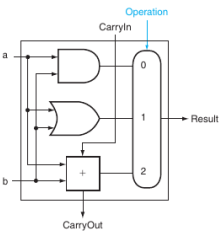
1-bit ALU



A	B	Cin	S	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

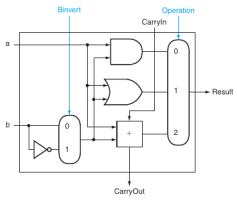
- Next is a full adder
- Carry input and carry output

1-bit ALU



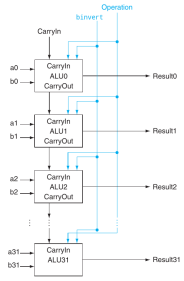
- Combine them in a package
- Multiplexer selects the operation among the inputs as AND, OR, ADD
- What if we want to subtract?

1-bit ALU



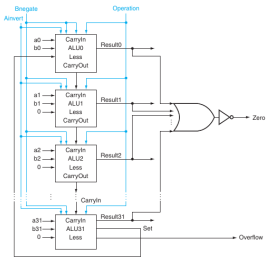
- What if we want to subtract?
- Add an invert option.
- a + (-b) = a – b
- -b is the 2's complement of the number.

32-bit ALU

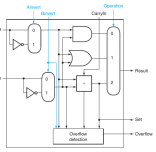


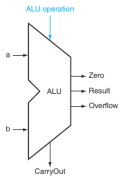
- We designed a very basic ALU.
- Expand it to 32-bit
- 1-bit ALU block for each bit
- Carry out is connected to next digit
- Operation bits are common

32-bit ALU



- Overflow detection
- When the result sign is not consistent with the input numbers
- Zero detection





Research

- How does a multiple input XOR gate work?
- Research ALU designs for different CPUs
- What's the difference between a CPU and an MCU?
- Write the assembly programs doing the same operation with the following C codes, using 8086 instruction set on EMU8086.

```
int sum = 0;
for (int i=0;i<=10;i++){
        sum=sum+i;
        }
```

```
int i, j, k, t;

        t=0;
        for(i=0;i<5;i++){
                t=t+1;
                for(j=0;j<5;j++){
                        t=t+1;
                        for(k=0;k<5;k++){
                                t=t+1;
                        }
                }
        }
```